Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 1

Uvod u OpenCV

Uvod

Tutorijali na predmetu Uvod u digitalnu obradu slike uvode osnovne klase iz OpenCV biblioteke i pokazuju primjere kako koristiti te klase i njihove metode, kako bi studenti što lakše mogli da riješe laboratorijske vježbe i zadatke za samostalan rad. Pretpostavlja se poznavanje osnova programiranja u C++ jeziku (iako će se dosta stvari ponoviti, tako da se studenti ne trebaju brinuti ukoliko su zaboravili neke stvari). Toplo se preporučuje studentima redovna vježba na predmetu, jer na ispitima dolaze problemi koji traže ne samo poznavanje materije već i izvježbanost u rješavanju nekih praktičnih problema. Prvi tutorijal se bavi osnovnim elementima OpenCV biblioteke kao i ponavljanjem nekih osnovnih stvari za programiranje u C++ jeziku.

OpenCV biblioteka

OpenCV (Open Source Computer Vision) je open-source biblioteka koja uključuje više stotina algoritama računarske vizije. OpenCV ima tzv. modularnu strukturu, što znači da paket koji ste prethodno instalirali na svoje računare sadrži više dijeljenih ili statičnih biblioteka. Kako se studenti budu susretali sa novim konceptima na predmetu, tako će se upoznavati sa novim modulima. U prvom tutorijalu se obrađuju moduli za sržnu funkcionalnost (eng. *core functionality*) kao i nekoliko funkcija iz sljedećih modula: `imcodecs.hpp`, `highgui.hpp` i `videoio.hpp`.

Modul za *core* funkcionalnost je kompaktan modul koji definira osnovne sturkture podataka, koje uključuju multi-dimenzionalni niz `cv::Mat` i osnovne funkcije koje ostali moduli koriste. Modul za obradu slike sadrži linearno i nelinearno filtriranje slike, geometrijske transformacije slike (kao npr. mijenjanje veličine slike), konverziju prostora boja, histograme, itd.

Sve klase i funkcije iz OpenCV biblioteke se stavljaju u imenik (eng. *namespace*) `cv`. Dakle, ukoliko je potrebno da pristupite nekoj funkciji ili klasi iz OpenCV biblioteke, to se može uraditi preko specifikatora `cv::` ili preko naredbe `using namespace cv;` (za kratke programe ovo je sasvim uredu koristiti, ali za veće programe takva naredba se ne preporučuje zbog toga što preko nje "uvozimo" cijeli imenik u program[1]).

Neka trenutna ili buduća OpenCV eksterna imena mogu biti u konfliktu sa standardnom ili drugim bibliotekama - u tom slučaju je potrebno koristiti eksplicitno specifikator `cv::` kako bi se konflikt riješio. Za primjere koji se budu navodili unutar tutorijala, autor tutorijala će uvijek koristiti specifikatore `cv::` i `std::` kako ne bi došlo do zabune.

Osnovna klasa za manipulaciju digitalnim slikama

Kao što je prethodno rečeno, osnovna klasa koja se koristi za učitavanje i za obradu slika je multi-dimenzionalni niz `cv::Mat`. Kako bi se koristila ova klasa, potrebno je uključiti zaglavlje *core* preko naredbe `#include "opencv2/core.hpp"` ili `#include <opencv2/core.hpp>`, a ukoliko

znamo da iz *core* modula koristimo samo klasu `cv::Mat` a ne i ostale funkcionalnosti, može se pisati i: `#include <opencv2/core/mat.hpp>` (može se i verizija sa navodnicima koristiti). Ovdje je potrebno istaći da ukoliko je verzija OpenCV-a koju ste instalirali npr. verzija 4, onda je potrebno uključiti zaglavlje na sljedeći način: `#include <opencv4/opencv2/core.hpp>`.

Bitno je naglasiti da klasa `cv::Mat`, kao i ostale slične klase unutar C++ jezika (kao npr. `std::vector` klasa), sadrži destruktor koji dealocira memoriju koju je instanca klase `cv::Mat` zauzela u toku svog postojanja, tako da nije potrebno voditi računa o dealokaciji memorije. Ono što je specifično za `cv::Mat` klasu je da prilikom dodjele instanci ove klase, vrši se plitka a ne duboka kopija podataka. Dakle, ukoliko se izvrši dodjela tipa `cv::Mat B = A;` gdje je `A` tipa `cv::Mat`, onda obje instance pokazuju na istu strukturu. Ukoliko želite to izbjeći, potrebno je da koristite metodu `cv::Mat::clone` koja pravi duboku kopiju instance. Ako je potrebno da ručno oslobodite memoriju neke instance klase `cv::Mat`, možete koristiti funkciju `.release()`.

U nastavku će biti pokazano na primjeru kako učitati proizvoljnu sliku, prikazati je u prozoru programa i sačuvati je na novoj lokaciji pod novim imenom.

Kreiranje prvog programa u OpenCV-u

```
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/core/mat.hpp>
#include <opencv4/opencv2/highgui.hpp>

int main() {

    // Deklaracija prve slike koju ucitavamo:
    cv::Mat prva_slika;

    // Ucitavanje prve slike:
    prva_slika = cv::imread("put_do_slike_iz_priloga/cat.jpeg",
        cv::IMREAD_COLOR);

    // Prikaz prve slike u prozoru programa:
    cv::imshow("Prva ucitana slika", prva_slika);

    // Cekamo unos od korisnika da bismo prestali prikazivati sliku:
    while(cv::waitKey() != 'q');

    // Cuvanje prve slike na novoj lokaciji pod novim imenom:
    cv::imwrite("../prva_slika.jpeg", prva_slika);

    return 0;

}
```

U gornjem programu uključili smo tri modula - `imgcodecs.hpp`, `core/mat.hpp` i `highgui.hpp`. Modul `core/mat.hpp` je već prethodno bio objašnjen, tako da tu se nećemo zadržavati. Važno je istaći da su funkcije `cv::imread` i `cv::imrwrite` iz modula `imgcodecs.hpp`, a funkcije `cv::imshow` i `cv::waitKey()` iz modula `highgui.hpp`.

Funkcija `cv::imread` prima dva parametra; prvi parametar je tipa `const String&` i predstavlja put do slike koju želimo učitati, dok je drugi parametar cjelobrojnog tipa koji treba navesti posebnim *flag*-om koji je definiran u OpenCV-u. U toku nastave, najčešće korišteni *flag*-ovi će biti `cv::IMREAD_COLOR` i `cv::IMREAD_GRAYSCALE`, koji učitavaju sliku u boji i u tonovima sive, respektivno.

Funkcija `cv::imshow` pravi novi prozor na ekranu i prima dva parametra - prvi parametar je tipa `const String&` i predstavlja naziv novootvorenog prozora, dok je drugi parametar tipa `InputArray` koji je u ovom slučaju `cv::Mat` tipa i predstavlja sliku koju smo prethodno učitali.

Ovaj prozor će da bude na ekranu dokle god korisnik ne unese "q", što je postignuto sa funkcijom `cv::waitKey()` unutar `while` petlje. Da nema linije 18, onda ne bismo uopšte vidjeli prozor sa slikom jer je on otvoren sve dok program radi - zbog toga čekamo unos sa tastature da bismo prešli na narednu funkciju.

Posljednja funkcija je `cv::imwrite` koja prima dva parametra: parametar tipa `const String&` koji predstavlja put do mjesta gdje želimo sačuvati učitanu sliku i drugi parametar tipa `InputArray` koji je ovdje `cv::Mat` tipa i predstavlja našu učitanu sliku. U ovom primjeru je put napisan kao `"../"` - ovo znači da će se slika sačuvati na lokaciji gdje se nalazi kreirani projekat.

Treba naglasiti da svaki put kada se učitava i čuva slika da je potrebno pisati i naziv i ekstenziju slike unutar puta koji je naveden - česta greška je navođenje pogrešne ekstenzije slike. Srećom, na ovom predmetu slike su uglavnom sa ekstenzijom .jpg, .jpeg i .png tako da ne bi trebalo biti prevelike zabune oko odgovarajuće ekstenzije.

Manipulacija video datotekama u OpenCV-u

Da bismo mogli učitati video snimak sa kamere ili sa video datotetke ili sekvenci slika u OpenCV-u, potrebno je koristiti posebnu klasu za učitavanje videa - `cv::VideoCapture`. Ova klasa se nalazi unutar modula `videoio.hpp` i u nastavku će biti pokazan osnovni primjer za učitavanje video datoteke i spašavanje video datoteke u novu datoteku.

```
1  #include <opencv4/opencv2/core/mat.hpp>
2  #include <opencv4/opencv2/highgui.hpp>
3  #include <opencv4/opencv2/videoio.hpp>
4
5  int main() {
6
7      // Ucitavanje videa u instancu klase cv::VideoCapture:
8      cv::VideoCapture originalni_video("put_do_videa/video.mp4");
9
10     // Varijabla tipa cv::Mat koju cemo koristiti za ucitavanje frejmova iz
           originalnog videa:
11     cv::Mat frame;
12
13     // Prikaz videa u novom prozoru:
14
15     // Instance klase cv::VideoCapture se ponasaju slicno kao instance
           input streamova (std::cin):
16     originalni_video >> frame;
17
18     // Dokle god ucitavamo frameove, neka se vrti while petlja (dakle, dok
           ne dodjemo do kraja videa):
19     while(!frame.empty()) {
20
21         // Prikazujemo frame u novom prozoru:
22         cv::imshow("Ucitani video", frame);
23
24         // Ukoliko pritisnemo 'q' program izlazi iz petlje:
25         if(cv::waitKey() == 'q') break;
26
27         // Ucitaj novi frame:
28         originalni_video >> frame;
29     }
30
31     // Zatvaramo video file ili kameru ovom naredbom:
32     originalni_video.release();
33
34     return 0;
35 }
```

U gornjem primjeru je prikazano kako učitati video iz video datoteke i prikazati ga na ekranu u novom prozoru. U liniji 8 je prikazano instanciranje objekta tipa `cv::VideoCapture`; vidimo da je potrebno varijabli samo proslijediti put do željenog videa kako bismo pozvali konstruktor. Pored objekta tipa `cv::VideoCapture` potreban nam je i objekat tipa `cv::Mat` kako bismo mogli video učitavati *"frame by frame"* - možemo posmatrati da se svaki video sastoji od niza slika koje dolaze jedna iza druge i kako se smjenjuju naše oko to percipira kao kretanje objekta/objekata na ekranu.

Instance klase `cv::VideoCapture` se ponašaju slično kao objekti `std::cin` iz `iostream` biblioteke. Tako da možemo vidjeti po linijama 16 i 28 iz gornjeg primjera da je potrebno samo napisati konstrukciju tipa `ucitani_video >> frame` kako bismo učitali naredni *frame* iz videa. U liniji 19 počinje `while` petlja koja nam omogućava nanovo učitavanje sljedećeg *frame*-a i prikazivanje svakog unutar prozora programa (isto kao što smo prikazivali pojedinačne slike u prethodnom primjeru).

Ono što je bitno naglasiti je da kako bismo prešli na naredni *frame*, potrebno je da korisnik pritišće bilo koju tipku na tastaturi osim tipke "q" - koju smo ovdje stavili kao opciju za prekidanje prikaza videa prije nego što se dođe do posljednjeg *frame*-a (tj. kraja videa).

Na kraju, u liniji 32 naredbom `originalni_video.release()` zatvaramo otvoreni video ili kameru koju koristimo. Ova metoda se automatski zove ukoliko iskoristimo funkciju `cv::VideoCapture::open` kako bismo novi video otvorili ali smjestili ga u istu varijablu i ukoliko se pozove `cv::VideoCapture` destruktor. U ovom primjeru naredba u liniji 32 jeste suvišna (sjetimo se da na kraju programa OpenCV se brine da se pozovu svi potrebni destruktori unutar programa), ali ova naredba može koristiti u nekim drugim situacijama, tako da je ostavljena unutar programa kao primjer.

Ukoliko želimo da učitavamo video sa web kamere, onda je u prethodnom primjeru potrebno zamjeniti u liniji 8 put do videa sa nulom: `cv::VideoCapture originalni_video(0)`. U tom slučaju će se u novootvorenom prozoru koji prikazuje video prikazivati trenutni snimak sa vaše video kamere (ukoliko je imate, naravno) - ali opet će biti potrebno pritiskati na tastaturi neku od tipki da bi se trenutni *frame* mogao prikazati unutar prozora, a pritiskom na tipku "q" `while` petlja se zaustavlja i zatvara se prozor i time završava program.

Još jedna stvar koja je potrebna je da naučimo je kako snimiti video u novu datoteku, a to će biti pokazano u sljedećem primjeru.

```
1  #include <opencv4/opencv2/core/mat.hpp>
2  #include <opencv4/opencv2/highgui.hpp>
3  #include <opencv4/opencv2/videoio.hpp>
4
5  int main() {
6
7      // Instanciranje objekta tipa cv::VideoCapture ali preko web kamere:
8      cv::VideoCapture kamera(0);
9
10     // Treba nam opet frame:
11     cv::Mat frame;
12
13     // Potrebna nam je sirina i visina videa za instanciranje novog tipa
14     // za snimanje snimka sa kamere u datoteku:
15     int sirina_videa = kamera.get(cv::CAP_PROP_FRAME_WIDTH);
16     int visina_videa = kamera.get(cv::CAP_PROP_FRAME_HEIGHT);
17
18     // Kako bismo mogli snimiti video sa kamere, potrebna nam je nova klasa:
19     cv::VideoWriter snimak_sa_kamere("../snimak_sa_kamere.avi",
           cv::VideoWriter::fourcc('M', 'J', 'P', 'G'), 10,
           cv::Size(sirina_videa, visina_videa));
20
21     // Ucitavanje narednog frame-a unutar uslova petlje i provjera da li je
```

```
        zadnji frame
22      // U ovom kontekstu to nema smisla jer je u pitanju video koji se
            direktno sa
23      // web kamere snima, ali moze se isto iskoristiti ako se ucitava video
24      // iz datoteke:
25      while(kamera >> frame, !frame.empty()) { // Koristenje sejtan operatora
            se ne preporucuje!
26
27          // Upisivanje frame-a u novi snimak:
28          snimak_sa_kamere.write(frame);
29
30          // Prikaz trenutnog frame-a:
31          cv::imshow("Trenutni frame", frame);
32
33          // Ukoliko se pritisne "q", break-a se while petlja:
34          if(cv::waitKey() == 'q') break;
35      }
36
37      // Slicno kao u prethodnom primjeru, iskoritimo .release() funkciju za
38      // zatvaranje videa:
39      snimak_sa_kamere.release();
40      kamera.release();
41
42      return 0;
43  }
```

Za upis u video datoteku smo morali koristiti `cv::VideoWriter` klasu. Konstruktor ove klase prima sljedeće parametre: varijablu tipa `const String&` koja predstavlja ime datoteke u koju ćemo snimiti (upisati) video (obavezno dodati odgovarajući nastavak); varijablu tipa `int fourcc` koja predstavlja 4-karakterni kod kodeka koji se koristi da bi se *frame*-ovi kompresovali - mi smo koristili `cv::VideoWriter::fourcc('M', 'J', 'P', 'G')` koja je *motion-jpeg* kodek; varijablu tipa `double` koja predstavlja broj *frame*-ova po sekundi; parametar tipa `cv::Size(width, height)` koji predstavlja veličinu video *frame*-ova; i na kraju predefinisani parametar tipa `bool isColor = true` koji ako je nenulti (automatski jeste nenulti) onda enkoder očekuje i enkodirat će *frame*-ove u boji, a u suprotnom će raditi sa *grayscale frame*-ovima (trenutno ovaj parametar je jedino podržan u Windows operativnom sistemu).

Učitavnje širine i visine videa smo uradili preko funkcije `.get()` u linijama 15 i 16, tako što smo proslijedili odgovarajući *flag* tipa `enum` funkciji `cv::CAP_PROP_FRAME_WIDTH` i `cv::CAP_PROP_FRAME_HEIGHT`.

Nakon toga, procedura za upisivanje videa je slična kao i za prikaz svakog novog *frame*-a u prozoru na ekranu, gdje se unutar `while` petlje koristi funkcija `.write()` nad objektom `snimak_sa_kamere` gdje se prosljeđuje objekat `frame`. Petlja se prekida pritiskom na tipku "q" ili ukoliko je učitani *frame* prazan (što se ne može desiti ukoliko se učitava video sa web kamere ukoliko je ona upaljena, ali može se desiti ukoliko snimamo video iz neke video datoteke).

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[2].

Literatura

[1] prof. dr Željko Jurić. "Predavanje 1". *Predavanja iz predmeta Tehnike Programiranja* (2023.).

[2] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.